

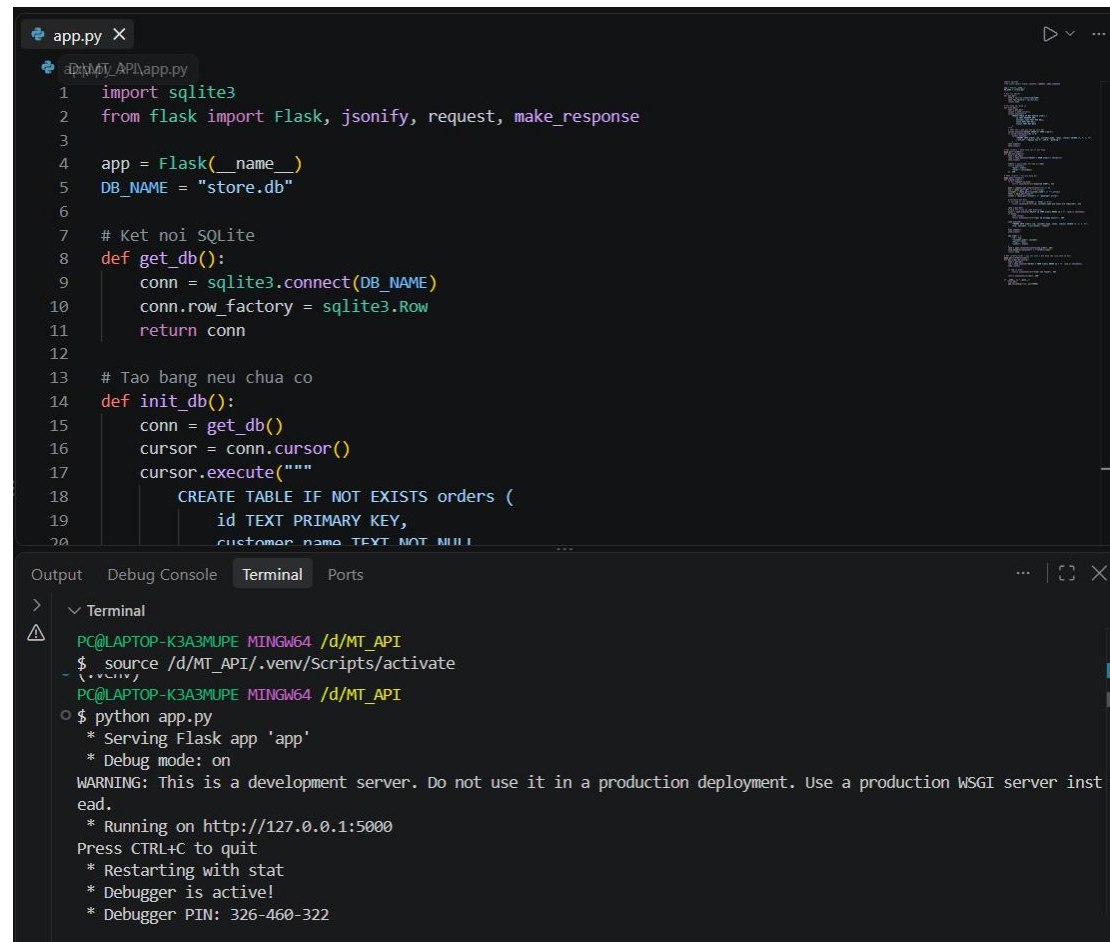
Bài 1: LƯU TRỮ VỚI SQLITE

Thay biến mảng cũ bằng database SQLite (store.db) để dữ liệu không bị mất khi tắt server.

Tạo bảng orders gồm các cột: id, customer_name, total, status.

Viết các API: lấy danh sách, thêm đơn mới và xem chi tiết theo mã đơn (GET /orders/<oid>).

Xử lý các mã trạng thái: 200 (lấy dữ liệu ok), 201 (tạo mới xong, có gắn header Location), 404 (không tìm thấy), 409 (trùng id đơn), 422 (thiếu thông tin bắt buộc).



```
app.py X
d:\MT_API\app.py
1 import sqlite3
2 from flask import Flask, jsonify, request, make_response
3
4 app = Flask(__name__)
5 DB_NAME = "store.db"
6
7 # Kết nối SQLite
8 def get_db():
9     conn = sqlite3.connect(DB_NAME)
10    conn.row_factory = sqlite3.Row
11    return conn
12
13 # Tạo bảng nếu chưa có
14 def init_db():
15     conn = get_db()
16     cursor = conn.cursor()
17     cursor.execute("""
18         CREATE TABLE IF NOT EXISTS orders (
19             id TEXT PRIMARY KEY,
20             customer_name TEXT NOT NULL,
21             total TEXT NOT NULL,
22             status TEXT NOT NULL
23         )
24     """)
25     conn.commit()
26     conn.close()
27
28 # Khởi tạo database
29 init_db()
30
31 # API lấy danh sách đơn hàng
32 @app.get('/orders')
33 def get_orders():
34     conn = get_db()
35     cursor = conn.cursor()
36     cursor.execute("SELECT * FROM orders")
37     orders = cursor.fetchall()
38     conn.close()
39     return jsonify([dict(zip(['id', 'customer_name', 'total', 'status'], o)) for o in orders])
40
41 # API thêm đơn hàng mới
42 @app.post('/orders')
43 def create_order():
44     data = request.get_json()
45     if not data or 'id' not in data or 'customer_name' not in data or 'total' not in data or 'status' not in data:
46         return jsonify({"error": "Thiếu thông tin bắt buộc"}), 422
47     conn = get_db()
48     cursor = conn.cursor()
49     cursor.execute("SELECT * FROM orders WHERE id = ?", (data['id'],))
50     if cursor.fetchone():
51         return jsonify({"error": "Trùng id đơn"}), 409
52     cursor.execute("INSERT INTO orders (id, customer_name, total, status) VALUES (?, ?, ?, ?)",
53                   (data['id'], data['customer_name'], data['total'], data['status']))
54     conn.commit()
55     conn.close()
56     return jsonify({"id": data['id'], "customer_name": data['customer_name'], "total": data['total'], "status": data['status']}), 201
57
58 # API xem chi tiết đơn hàng
59 @app.get('/orders/<oid>')
60 def get_order_by_id(oid):
61     conn = get_db()
62     cursor = conn.cursor()
63     cursor.execute("SELECT * FROM orders WHERE id = ?", (oid,))
64     order = cursor.fetchone()
65     conn.close()
66     if order:
67         return jsonify(dict(zip(['id', 'customer_name', 'total', 'status'], order)))
68     return jsonify({"error": "Không tìm thấy đơn hàng"}), 404
69
70 if __name__ == '__main__':
71     app.run(debug=True)
```

Output Debug Console Terminal Ports

> Terminal

```
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ source /d/MT_API/.venv/Scripts/activate
- .venv
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 326-460-322
```

```

PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ curl http://localhost:5000/orders
{
  "data": [
    {
      "customer_name": "Nguyen Van A",
      "id": "ord_01",
      "status": "pending",
      "total": 150.0
    }
  ],
  "total": 1
}
(.venv)
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ curl http://localhost:5000/orders/ord_01
{
  "customer_name": "Nguyen Van A",
  "id": "ord_01",
  "status": "pending",
  "total": 150.0
}
(.venv)

```

```

PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ curl -i -X POST http://localhost:5000/orders -H "Content-Type: application/json" -d '{"id":
"ord_02", "customer_name": "Tran Van B", "total": 220.5}'
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Mon, 21 Sep 2026 16:44:09 GMT
Content-Type: application/json
Content-Length: 95
Location: /orders/ord_02
Connection: close

{
  "customer_name": "Tran Van B",
  "id": "ord_02",
  "status": "pending",
  "total": 220.5
}
(.venv)

```

Bài 2: BÁO CÁO AUDIT GITHUB REST API

Em chọn GitHub REST API tại địa chỉ <https://api.github.com> để khảo sát 5 endpoint bằng curl. Mục đích là tìm hiểu cách GitHub sử dụng URI, HTTP method, status code và một số header trong REST API.

1. LẤY THÔNG TIN USER

Endpoint: GET /users/{username}

Method: GET

Status code:

200 OK: lấy thông tin user thành công.

404 Not Found: không tìm thấy user.

Header chính:

Accept: application/vnd.github+json

Response trả về thông tin của user dưới dạng JSON, ví dụ tên tài khoản, số repository, số người theo dõi.

Đánh giá:

Endpoint này có tính RESTful vì /users/{username} dùng để xác định tài nguyên, còn GET được dùng để lấy dữ liệu. GitHub cũng hỗ trợ cơ chế cache thông qua các HTTP header như ETag.

2. TẠO REPOSITORY MỚI

Endpoint: POST /user/repos

Method: POST

Status code:

201 Created: tạo repository thành công.

401 Unauthorized: chưa xác thực hợp lệ.

422 Unprocessable Entity: dữ liệu gửi lên không hợp lệ.

Header chính:

Authorization: Bearer <YOUR-TOKEN>

Content-Type: application/json

Accept: application/vnd.github+json

Request body được gửi dưới dạng JSON, trong đó có thể truyền tên repository, mô tả, trạng thái private và một số thông tin khác.

Đánh giá:

POST được dùng đúng mục đích là tạo tài nguyên mới. URI /user/repos biểu diễn nhóm repository của người dùng, còn thông tin cần tạo được gửi trong request body. Cách thiết kế này phù hợp với REST.

3. SỬA REPOSITORY

Endpoint: PATCH /repos/{owner}/{repo}

Method: PATCH

Status code:

200 OK: cập nhật thành công.

403 Forbidden: không đủ quyền.

404 Not Found: không tìm thấy repository.

422 Unprocessable Entity: dữ liệu gửi lên không hợp lệ.

Header chính:

Authorization: Bearer <YOUR-TOKEN>

Content-Type: application/json

Accept: application/vnd.github+json

Ví dụ có thể chỉ gửi description mới để thay đổi mô tả của repository.

Đánh giá:

PATCH phù hợp khi chỉ muốn cập nhật một phần thông tin của repository. URI vẫn xác định đúng repository cần sửa, còn PATCH cho biết đây là thao tác cập nhật. Đây là cách sử dụng phù hợp với REST.

4. XÓA REPOSITORY

Endpoint: DELETE /repos/{owner}/{repo}

Method: DELETE

Status code:

204 No Content: xóa thành công.

403 Forbidden: không đủ quyền.

404 Not Found: không tìm thấy repository.

Header chính:

Authorization: Bearer <YOUR-TOKEN>

Accept: application/vnd.github+json

Khi xóa thành công, GitHub trả về 204 No Content vì không cần gửi thêm dữ liệu trong response body.

Đánh giá:

DELETE được dùng đúng mục đích để xóa tài nguyên. Thao tác này có tính idempotent theo ý nghĩa của HTTP, tức là việc thực hiện lại yêu cầu xóa không làm xuất hiện thêm tài nguyên mới.

5. ĐÁNH DẤU SAO REPOSITORY

Endpoint: PUT /user/starred/{owner}/{repo}

Method: PUT

Status code:

204 No Content: star thành công.

401 Unauthorized: chưa xác thực.

403 Forbidden: không đủ quyền.

404 Not Found: không tìm thấy repository.

Header chính:

Authorization: Bearer <YOUR-TOKEN>

Accept: application/vnd.github+json

PUT ở đây được dùng để đặt repository vào trạng thái đã được người dùng star.

Đánh giá:

Endpoint này có tính idempotent. Nếu gửi cùng một request nhiều lần thì trạng thái cuối cùng của repository vẫn là đã được star.

6. BẢNG TỔNG HỢP

Chức năng: Lấy thông tin user

Method: GET

Endpoint: /users/{username}

Status thành công: 200 OK

Chức năng: Tạo repository

Method: POST

Endpoint: /user/repos

Status thành công: 201 Created

Chức năng: Sửa repository

Method: PATCH

Endpoint: /repos/{owner}/{repo}

Status thành công: 200 OK

Chức năng: Xóa repository

Method: DELETE

Endpoint: /repos/{owner}/{repo}

Status thành công: 204 No Content

Chức năng: Star repository

Method: PUT

Endpoint: /user/starred/{owner}/{repo}

Status thành công: 204 No Content

7. KẾT LUẬN

Qua 5 endpoint trên, có thể thấy GitHub REST API sử dụng khá rõ các nguyên tắc của REST. URI được dùng để xác định tài nguyên, HTTP method được dùng cho từng loại thao tác khác nhau và status code cho biết kết quả của request. Dữ liệu được trao đổi chủ yếu dưới dạng JSON và các thao tác thay đổi dữ liệu có cơ chế xác thực và phân quyền.

Theo Richardson Maturity Model, các endpoint được khảo sát thể hiện rõ Level 2 vì đã sử dụng Resources, HTTP Verbs và HTTP Status Codes. Trong phạm vi bài khảo sát này, chưa có đủ cơ sở để kết luận API đạt đầy đủ Level 3 với HATEOAS.

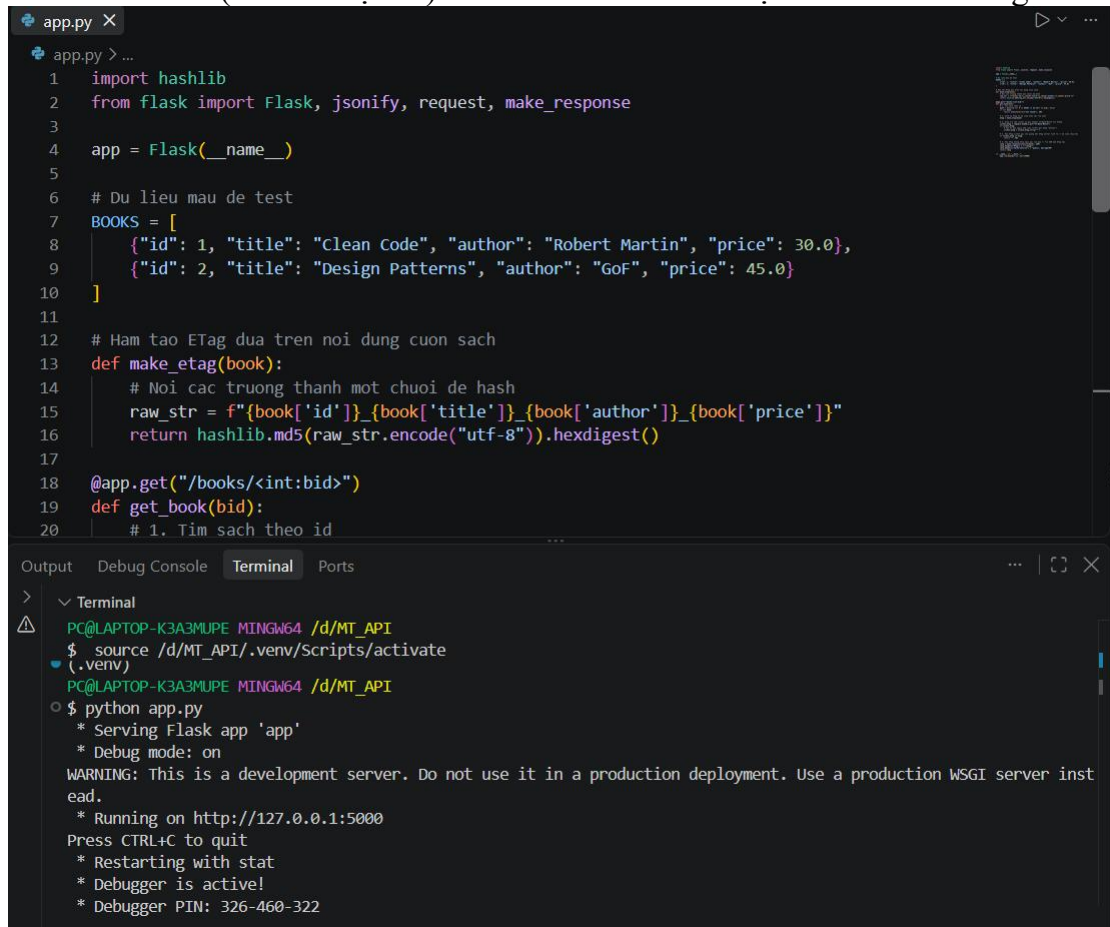
Bài 3: KIỂM TRA CACHE BẰNG ETAG

Dùng thư viện hashlib băm (MD5) nội dung cuốn sách thành một chuỗi mã ETag gắn vào response.

Khi client gửi lại mã này qua header If-None-Match, server lấy ra so sánh:

+Nếu mã khớp (dữ liệu chưa đổi): Trả mã 304, không kèm nội dung để đỡ tốn dung lượng mạng.

+Nếu mã khác (sách đã bị sửa): Trả mã 200 kèm dữ liệu mới và mã ETag mới.



```
app.py X
app.py > ...
1 import hashlib
2 from flask import Flask, jsonify, request, make_response
3
4 app = Flask(__name__)
5
6 # Du lieu mau de test
7 BOOKS = [
8     {"id": 1, "title": "Clean Code", "author": "Robert Martin", "price": 30.0},
9     {"id": 2, "title": "Design Patterns", "author": "GoF", "price": 45.0}
10 ]
11
12 # Ham tao ETag dua tren noi dung cuon sach
13 def make_etag(book):
14     # Noi cac truong thanh mot chuoai de hash
15     raw_str = f"{book['id']}_{book['title']}_{book['author']}_{book['price']}"
16     return hashlib.md5(raw_str.encode("utf-8")).hexdigest()
17
18 @app.get("/books/<int:bid>")
19 def get_book(bid):
20     # 1. Tim sach theo id
    ...

Output Debug Console Terminal Ports
>
Terminal
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ source /d/MT_API/.venv/Scripts/activate
(.venv)
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 326-460-322
```

```
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
```

```
$ curl -i http://localhost:5000/books/1
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Mon, 21 Sep 2026 17:27:39 GMT
Content-Type: application/json
Content-Length: 85
ETag: "7f5b6223786fad4ae56765f6d8fd6ba0"
Cache-Control: public, max-age=60
Connection: close
```

```
{
  "author": "Robert Martin",
  "id": 1,
  "price": 30.0,
  "title": "Clean Code"
}
```

```
(.venv)
```

```
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
```

```
$ curl -i -H 'If-None-Match: "a66f44d825c56c2214407b483c66f56a"' http://localhost:5000/books/1
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Mon, 21 Sep 2026 17:27:51 GMT
Content-Type: application/json
Content-Length: 85
ETag: "7f5b6223786fad4ae56765f6d8fd6ba0"
Cache-Control: public, max-age=60
Connection: close
```

```
{
  "author": "Robert Martin",
  "id": 1,
  "price": 30.0,
  "title": "Clean Code"
}
```

```
(.venv)
```